

Accelerometer Algorithm Inputs — What Signal Each Stage Matches Against

July 1, 2026

Purpose. Quick reference for what every accelerometer-only algorithm in this repo consumes as input, what it does in plain words, and how the trapezoid match in *segmentation* differs from the trapezoid match in *prediction*. Barometer paths are out of scope.

1 Common signal definitions

Every accelerometer algorithm starts from the raw 3-axis device-frame sample (a_x, a_y, a_z) at sampling rate f_s (typically 100 Hz). From it we derive three candidate scalar signals:

- $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$
Gravity *included*. Orientation-blind.
Used by: the GT editor (`src/data/gt_editor.py:227`) for visual annotation only. **No active algorithm consumes $|a|$.**
- a_z
Device-frame z -axis. **No active algorithm consumes a_z .**
- $a_{\text{vert}} = (a - \hat{g}) \cdot \frac{\hat{g}}{\|\hat{g}\|}$
Signed scalar projection of the acceleration vector onto the estimated gravity direction $\hat{g}$. Positive = up.
This is the signal every active accelerometer algorithm uses.

The gravity vector $\hat{g}$ is estimated from a stationary window: `estimate_gravity_stationary` in `src/utils/accelerometer_utils.py`. In segmentation the window is the first 0.5s of the buffer (`...template_match/fit_elevator_parameters/common.py:136`). In prediction the pre/post ride stationary windows are preferred, falling back to in-ride estimation, then to $|a|$ as a last resort (`trapezoid_accel/estimator.py:118-170`).

After projection the signal is detrended (slow rolling mean) and smoothed (rolling mean over `smooth_sec=0.4s`) to remove DC bias from orientation drift and hand-tremor:

$$a_{\text{vert}}^{(\text{smooth})}(t) = \text{RollMean}(a_{\text{vert}}(t) - \text{RollMean}_{\text{slow}}(a_{\text{vert}}(t)), w_{\text{smooth}}).$$

2 Segmentation — accelerometer path

Single active algorithm: **trapezoid matched-filter template match**, dispatched as `SegmentAlgorithm.ACC_TEMPLATE_MATCH`.

Files: `src/segmentation/algorithms/accelerometer_only/template_match/—matcher.py` (legacy learned-template path) and `check_grid_across_signal/detect.py` (active synthetic-grid path, wired to the editor UI).

Input. Smoothed a_{vert} over the *entire session* (`matcher.py:118`, `detect.py` via the shared preprocessing in `fit_elevator_parameters/common.py:135-144`). The matcher also computes an LPF velocity v_{lpf} by integrating a_{vert} , and the two scores are linearly fused (`matcher.py:125-127`); the active grid detector uses a_{vert} alone.

Template. Synthetic trapezoid kernel parameterised by half-width W and flat-fraction $f \in [0, 1]$:

$$\text{tpl}(t; t_c, W, f) = \begin{cases} 1 & |t - t_c| \leq fW \quad (\text{flat top}) \\ \frac{W - |t - t_c|}{W(1 - f)} & fW < |t - t_c| \leq W \quad (\text{linear ramp}) \\ 0 & |t - t_c| > W. \end{cases}$$

Grid: $30 \times 15 = 450$ templates over $W \in [0.3, 3.0]$ s, $f \in [0.05, 0.80]$, plus the $f = 0$ triangle row (`detect.py:130-143`).

What it does (problem type: *detection*). Slide every template across the whole signal using Pearson normalised cross-correlation (`ncc_slide`, `matcher.py:33-54`). Reduce the per-sample argmax across (W, f) to a peak score, then:

1. **Peak-pick** on unsigned R^2 with threshold `r2_peak_thresh=0.40` and a minimum matched amplitude $|A| \geq 0.20$ m/s².
2. **NMS dedup** within `nms_radius_s=1.0` s and same-sign suppression within 5 s.
3. **Pair** a positive (take-off) peak with a later negative (landing) peak, accept if their shared-shape joint $R^2 \geq 0.90$, the pair $|A| \geq 0.30$ m/s², and the plateau between the lobes is quiet (`quiet_middle_ratio=0.5`).

Output. A list of ride intervals $(t_{\text{start}}, t_{\text{end}}, \text{direction})$. No height value is produced.

3 Prediction — accelerometer paths

Two accelerometer-only predictors share the `predict_segment(ride, pre, post) -> PredictionOutput` contract dispatched by `Predictor` in `src/prediction/algorithms/predictor.py:29-43`.

3.1 ZUPT double-integration

File: `src/prediction/algorithms/accelerometer_only/zupt_accel/estimator.py`.

Input. a_{vert} over the *ride window*, plus the pre/post stationary windows used to estimate $\hat{g}$.

What it does.

1. Detect the active-motion sub-window by thresholding $|a_{\text{vert}}|$ (`_find_motion_window`, `estimator.py:50-65`).
2. Zero-force $a_{\text{vert}}(t) = 0$ outside that sub-window (`_zupt_integrate_windowed`, `estimator.py:68-79`).
3. Double-integrate $a_{\text{vert}} \rightarrow v \rightarrow h$, applying linear-drift correction so v returns to zero at the boundaries.

Trapezoid involvement. **None.** ZUPT does not match any template; it is a pure double-integration with zero-velocity-update bookends.

Output. Signed Δh in metres plus a theoretical σ from white-noise integration bounds.

3.2 Trapezoid pulse-pair fit

File: `src/prediction/algorithms/accelerometer_only/trapezoid_accel/estimator.py`;
kernel + grid search in `pulse_pair.py`.

Input. Detrended + smoothed a_{vert} over the *ride window* (`estimator.py:190-196`): same projection, same `smooth_sec=0.4s` rolling mean.

What it does (problem type: *parameter estimation*). Searches the same (W, f) trapezoid grid as segmentation (`pulse_pair.py:42`, 30×15). For every grid point and every candidate centre pair (t_{c1}, t_{c2}) it solves least-squares for a *shared* amplitude $|A|$ across the two lobes and scores by the joint $R^2 = \frac{1}{2}(R_1^2 + R_2^2)$ (`pulse_pair.py:42-62`). The argmax across (W, f, t_{c1}, t_{c2}) is kept. A second 4-parameter *joined-pulse* fit ($\Delta t_c = 2W$ forced) is run in parallel; the better joint- R^2 wins (`estimator.py:213-254`). A ZUPT fallback kicks in if both matched-filter fits collapse.

Output. Closed-form

$$\Delta h = \text{sign} \cdot |A| \cdot W \cdot (1 + f) \cdot (t_{c2} - t_{c1})$$

(`estimator.py:330-339`), plus a delta-method theoretical σ and a conformal-calibrated 90% CI half-width.

4 Segmentation vs. prediction — same trapezoid, different question

Both stages match the same trapezoid template family against the same projected signal a_{vert} . The difference is what they do with the match.

	Segmentation	Prediction (trapezoid pulse-pair)
Input scope	smoothed a_{vert} over the whole session	detrended+smoothed a_{vert} over one ride window
Template family	synthetic trapezoid (W, f) grid + $f=0$ triangle row	same (W, f) grid, shared-shape constraint across the lobe pair
Scoring	sliding Pearson NCC, per-sample peak score	exhaustive LS amplitude per (W, f, t_{c1}, t_{c2}) , joint R^2
What is found	peak <i>locations</i> (when does a ride happen?)	best-fit <i>parameters</i> $(W, f, A , t_{c1}, t_{c2})$
Problem class	detection	parameter estimation
Output	ride intervals (t_s, t_e, dir)	Δh in metres + CI half-width
Filters / gates	peak $R^2 \geq 0.40$, NMS, pair $R^2 \geq 0.90$, quiet-middle	quality score, R^2 thresholds by ride length, anchor-ratio, Δt_c floors

The trapezoid kernel itself lives in a single shared utility (`src/utils/trapezoid_template.py`, re-exported by both stages) so the template parametrisation is identical by construction.

5 One-line summary

All active accelerometer algorithms — the segmentation matched-filter, the ZUPT predictor, and the trapezoid pulse-pair predictor — consume a_{vert} (gravity-projected vertical acceleration), not $|a|$. The trapezoid template appears in both stages, but with different roles: segmentation uses it as a *detector* (sliding NCC over the whole session to find rides), and prediction uses it as an *estimator* (exhaustive LS fit inside a single ride to recover Δh).

Implication for editor graphs. The GT editor (`src/data/gt_editor.py`) currently plots $|a|$, which no active algorithm uses. The segmentation editor (`src/segmentation/algorithms/editor.py:118-121`) and the Streamlit segmentation step (`src/pipelines/streamlit/step3_segmentation.py:199-209`) both already plot a_{vert} (with its smoothed overlay), which is the signal the matched filter actually scores against. Harmonising the three to display a_{vert} is a follow-up; this report is the reference for the decision.

6 Vertical-acceleration reconstruction from accel + gyro

Motivation. Every algorithm above consumes $a_{\text{vert}} = (a - \hat{g}) \cdot \hat{g}$, the accelerometer projected onto a *single* gravity vector $\hat{g}$ estimated from the pre/post stationary windows (`estimate_gravity_stationary`). That assumes the phone’s orientation is fixed for the whole ride. When the phone rotates mid-ride, $\hat{g}$ is stale and a slice of the 9.81 m/s^2 gravity term leaks into a_{vert} as a time-varying artefact that DC/drift removal cannot kill — it blurs the trapezoid and, because ZUPT double-integrates, a 5° tilt error ($\approx 0.04 \text{ m/s}^2$) becomes $\approx 1.9 \text{ m}$ over a 10 s ride. Reconstruction tracks the gravity direction $\hat{g}(t)$ with the gyroscope instead.

Pipeline. Code: `src/physics/reconstruct_az/`, invoked as the first line of `Predictor.predict / Segmenter.detect` via the one-liner `reconstruct_az(type, acc, gyro)` (config field `reconstruct`; `"none"` = today’s behaviour). Each filter estimates a body-to-world orientation quaternion $q(t)$, seeded from the opening stationary window, then

$$a_{\text{world}}(t) = R(q(t)) a(t), \quad a_{\text{vert}}(t) = a_{\text{world},z}(t) - g.$$

To keep the downstream gravity projection unchanged, the predictors/detector are actually fed the *virtually-flat-phone* signal ($a_{\text{world},x}, a_{\text{world},y}, a_{\text{world},z} + g$) — gravity kept on $+z$ — so `estimate_gravity_stationary` still recovers $|g| \approx 9.81$ and the projection returns the true signed vertical accel.

The five filters. All run in IMU mode (accel a , gyro ω , both body-frame; no magnetometer — yaw is irrelevant for the vertical channel). Shared quantities: normalised accel $\hat{a} = a/|a|$; predicted gravity direction in the body frame $\hat{v} = R(q)^\top \hat{z}$; and an *accel-trust* weight $w = \exp(-\frac{1}{2}((|a| - g)/(\tau g))^2)$ that down-weights the accelerometer correction during the ride’s acceleration pulses (when $|a| \neq g$), so ride acceleration is never mistaken for tilt.

1. **Complementary.** Gyro integration $q \leftarrow q \otimes \exp(\frac{1}{2}\omega \Delta t)$, then a partial rotation of $\hat{a}$ onto $\hat{z}$: $q \leftarrow \text{slerp}(1, \delta q; \alpha w) \otimes q$, with δq the shortest arc from $R(q)\hat{a}$ to $\hat{z}$.

2. **Mahony** (PI on $SO(3)$, gyro-bias). Error $e = \hat{a} \times \hat{v}$; bias $b \leftarrow b - k_I w e \Delta t$; $\dot{q} = \frac{1}{2}q \otimes (\omega - b + k_P w e)$.
3. **Madgwick** (gradient descent). With $f = R(q)^\top \hat{z} - \hat{a}$ and Jacobian J , $\dot{q} = \frac{1}{2}q \otimes \omega - \beta w \frac{J^\top f}{\|J^\top f\|}$.
4. **Valenti / AQUA** (algebraic + adaptive gain). Predict with the gyro, form the algebraic Δq_{acc} (shortest arc $R(q)\hat{a} \rightarrow \hat{z}$), and apply it with a gain that ramps *linearly to zero* once $||a| - g|/g$ exceeds a threshold: $q \leftarrow \text{slerp}(1, \Delta q_{\text{acc}}; \alpha) \otimes q$.
5. **ESKF** (error-state Kalman). Nominal (q, b) propagated by the gyro; error state $(\delta\theta, \delta b)$ tracked with covariance P . Accelerometer update uses $H = [[\hat{v}]_\times \ 0]$, measurement noise $R = (\sigma_a/w)^2 I$, then injects $q \leftarrow q \otimes [1, \frac{1}{2}\delta\theta]$ and resets. Estimating b online holds attitude through long low-excitation cruise phases.

Primary references (`src/physics/reconstruct_az/references/`): Euston et al. 2008 (complementary), Mahony et al. 2008, Madgwick 2010, Valenti et al. 2015, Solà 2017.

Does it help? (measured). Tables 1 and 2 score every variant on the real elevator set, produced by `scripts/reconstruction_comparison_report.py`. Reconstruction sharply reduces prediction error (best filter in **bold**) and removes segmentation false positives; the gain is largest on rides where the phone was rotated (Fig. 3).

Table 1: Prediction accuracy by reconstruction (ZUPT and trapezoid).

ZUPT					
Reconstruction	MAE (m)	RMSE (m)	Cov@90	≤ 1.5 m	med CI (m)
none	2.545	18.827	0.919	0.758	2.256
Complementary	1.075	3.246	0.911	0.871	1.252
Mahony	1.068	3.238	0.916	0.869	1.237
Madgwick	1.476	10.096	0.924	0.868	1.258
Valenti	1.075	3.243	0.913	0.869	1.215
ESKF	1.083	3.074	0.908	0.874	1.165
TRAPEZOID					
Reconstruction	MAE (m)	RMSE (m)	Cov@90	≤ 1.5 m	med CI (m)
none	1.734	11.313	0.835	0.848	0.894
Complementary	0.988	2.853	0.863	0.879	0.892
Mahony	0.943	2.734	0.864	0.880	0.892
Madgwick	1.173	3.452	0.850	0.871	0.892
Valenti	0.959	2.744	0.861	0.879	0.892
ESKF	0.964	2.786	0.866	0.884	0.892

Table 2: Segmentation accuracy by reconstruction (ACC template match).

Reconstruction	n_{gt}	detect	miss	FP	F1*	IoU-F1	recall	prec
none	627	498	116	43	0.847	0.726	0.794	0.907
Complementary	627	540	67	10	0.902	0.742	0.861	0.947
Mahony	627	550	67	8	0.921	0.774	0.877	0.970
Madgwick	627	511	91	92	0.803	0.660	0.815	0.792
Valenti	627	531	79	12	0.898	0.739	0.847	0.957
ESKF	627	524	86	43	0.860	0.715	0.836	0.885

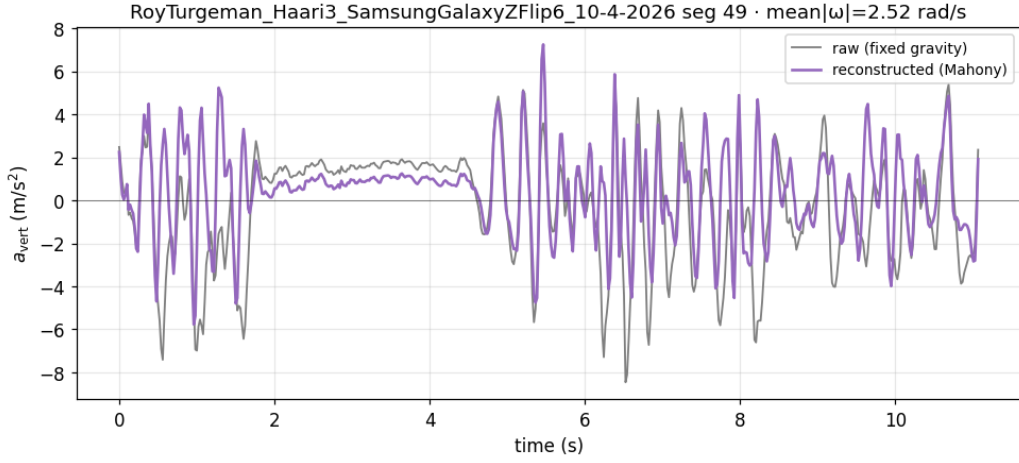


Figure 1: Raw fixed-gravity a_{vert} vs. reconstructed vertical acceleration on a rotating ride. The raw trace carries a tilt-induced bias in the calm mid-ride section that reconstruction removes.

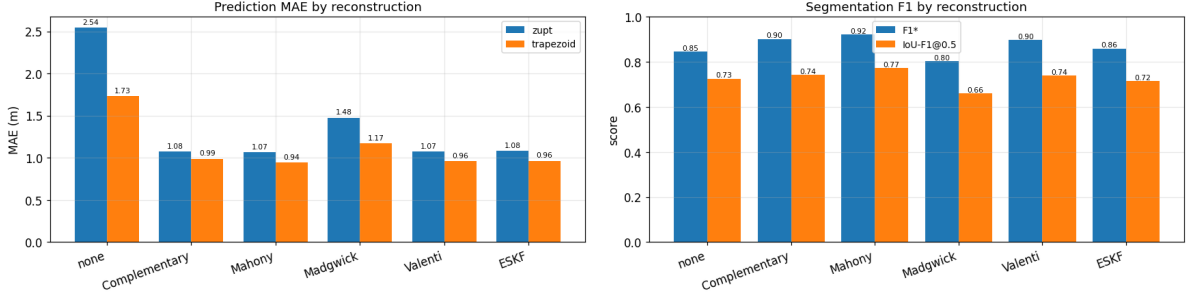


Figure 2: Left: prediction MAE by reconstruction. Right: segmentation F1 / IoU-F1 by reconstruction.

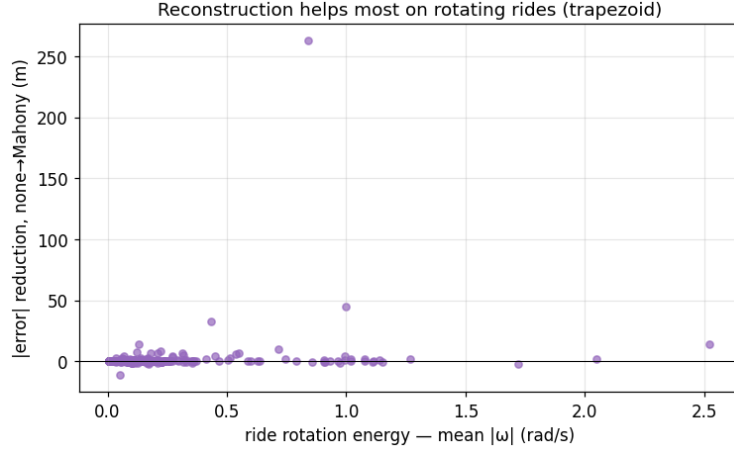


Figure 3: Per-ride rotation energy (mean $|\omega|$) vs. the trapezoid $|\text{error}|$ reduction from reconstruction — the benefit concentrates on the most-rotated rides.

Metrics that help. Beyond MAE / F1, three signals gauge reconstruction quality: the *accel-trust* weight w (fraction of the ride where the accelerometer is a usable gravity reference), per-ride *rotation energy* $\langle |\omega| \rangle$ (how much reconstruction can possibly help), and agreement of $q(t)$ with the phone’s own fused ORI quaternion (a free on-device reference).